

12.3 Studying Sorting through an Algorithmic Lens

Recapping our discussions on sorting to this point, we have described several methods with either a worst case or expected running time of $O(n \log n)$ on an input sequence of size n . These methods include merge-sort and quick-sort, described in this chapter, as well as heap-sort (Section 9.4.2). In this section, we will study sorting as an algorithmic problem, addressing general issues about sorting algorithms.

12.3.1 Lower Bound for Sorting

A natural first question to ask is whether we can sort any faster than $O(n \log n)$ time. Interestingly, if the computational primitive used by a sorting algorithm is the comparison of two elements, this is in fact the best we can do—comparison-based sorting has an $\Omega(n \log n)$ worst-case lower bound on its running time. (Recall the notation $\Omega(\cdot)$ from Section 4.3.1.) To focus on the main cost of comparison-based sorting, let us only count comparisons, for the sake of a lower bound.

Suppose we are given a sequence $S = (x_0, x_1, \dots, x_{n-1})$ that we wish to sort, and assume that all the elements of S are distinct (this is not really a restriction since we are deriving a lower bound). We do not care if S is implemented as an array or a linked list, for the sake of our lower bound, since we are only counting comparisons. Each time a sorting algorithm compares two elements x_i and x_j (that is, it asks, “is $x_i < x_j$?”), there are two outcomes: “yes” or “no.” Based on the result of this comparison, the sorting algorithm may perform some internal calculations (which we are not counting here) and will eventually perform another comparison between two other elements of S , which again will have two outcomes. Therefore, we can represent a comparison-based sorting algorithm with a decision tree T (recall Example 8.5). That is, each internal node v in T corresponds to a comparison and the edges from position v to its children correspond to the computations resulting from either a “yes” or “no” answer. It is important to note that the hypothetical sorting algorithm in question probably has no explicit knowledge of the tree T . The tree simply represents all the possible sequences of comparisons that a sorting algorithm might make, starting from the first comparison (associated with the root) and ending with the last comparison (associated with the parent of an external node).

Each possible initial order, or **permutation**, of the elements in S will cause our hypothetical sorting algorithm to execute a series of comparisons, traversing a path in T from the root to some external node. Let us associate with each external node v in T , then, the set of permutations of S that cause our sorting algorithm to end up in v . The most important observation in our lower-bound argument is that each external node v in T can represent the sequence of comparisons for at most one permutation of S . The justification for this claim is simple: If two different

permutations P_1 and P_2 of S are associated with the same external node, then there are at least two objects x_i and x_j , such that x_i is before x_j in P_1 but x_i is after x_j in P_2 . At the same time, the output associated with v must be a specific reordering of S , with either x_i or x_j appearing before the other. But if P_1 and P_2 both cause the sorting algorithm to output the elements of S in this order, then that implies there is a way to trick the algorithm into outputting x_i and x_j in the wrong order. Since this cannot be allowed by a correct sorting algorithm, each external node of T must be associated with exactly one permutation of S . We use this property of the decision tree associated with a sorting algorithm to prove the following result:

Proposition 12.4: *The running time of any comparison-based algorithm for sorting an n -element sequence is $\Omega(n \log n)$ in the worst case.*

Justification: The running time of a comparison-based sorting algorithm must be greater than or equal to the height of the decision tree T associated with this algorithm, as described above. (See Figure 12.15.) By the argument above, each external node in T must be associated with one permutation of S . Moreover, each permutation of S must result in a different external node of T . The number of permutations of n objects is $n! = n(n-1)(n-2) \cdots 2 \cdot 1$. Thus, T must have at least $n!$ external nodes. By Proposition 8.7, the height of T is at least $\log(n!)$. This immediately justifies the proposition, because there are at least $n/2$ terms that are greater than or equal to $n/2$ in the product $n!$; hence,

$$\log(n!) \geq \log \left(\left(\frac{n}{2} \right)^{\frac{n}{2}} \right) = \frac{n}{2} \log \frac{n}{2},$$

which is $\Omega(n \log n)$. ■

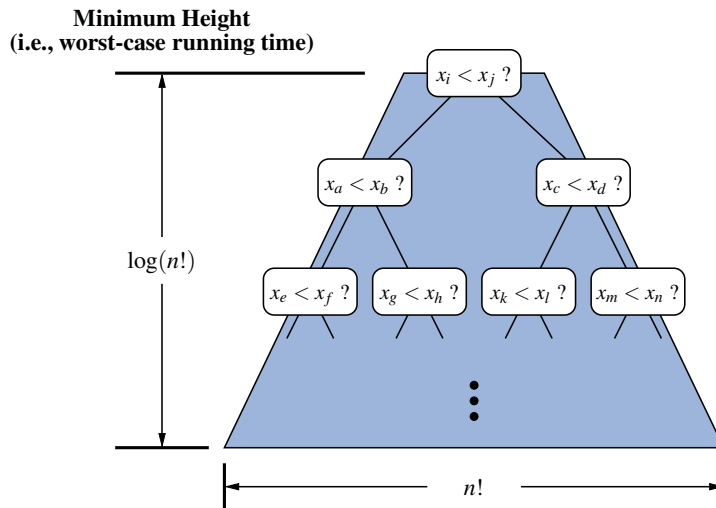


Figure 12.15: Visualizing the lower bound for comparison-based sorting.

12.3.2 Linear-Time Sorting: Bucket-Sort and Radix-Sort

In the previous section, we showed that $\Omega(n \log n)$ time is necessary, in the worst case, to sort an n -element sequence with a comparison-based sorting algorithm. A natural question to ask, then, is whether there are other kinds of sorting algorithms that can be designed to run asymptotically faster than $O(n \log n)$ time. Interestingly, such algorithms exist, but they require special assumptions about the input sequence to be sorted. Even so, such scenarios often arise in practice, such as when sorting integers from a known range or sorting character strings, so discussing them is worthwhile. In this section, we will consider the problem of sorting a sequence of entries, each a key-value pair, where the keys have a restricted type.

Bucket-Sort

Consider a sequence S of n entries whose keys are integers in the range $[0, N - 1]$, for some integer $N \geq 2$, and suppose that S should be sorted according to the keys of the entries. In this case, it is possible to sort S in $O(n + N)$ time. It might seem surprising, but this implies, for example, that if N is $O(n)$, then we can sort S in $O(n)$ time. Of course, the crucial point is that, because of the restrictive assumption about the format of the elements, we can avoid using comparisons.

The main idea is to use an algorithm called **bucket-sort**, which is not based on comparisons, but on using keys as indices into a bucket array B that has cells indexed from 0 to $N - 1$. An entry with key k is placed in the “bucket” $B[k]$, which itself is a sequence (of entries with key k). After inserting each entry of the input sequence S into its bucket, we can put the entries back into S in sorted order by enumerating the contents of the buckets $B[0], B[1], \dots, B[N - 1]$ in order. We describe the bucket-sort algorithm in Code Fragment 12.7.

Algorithm bucketSort(S):

Input: Sequence S of entries with integer keys in the range $[0, N - 1]$

Output: Sequence S sorted in nondecreasing order of the keys

let B be an array of n sequences, each of which is initially empty

for each entry e in S **do**

 let k denote the key of e

 remove e from S and insert it at the end of bucket (sequence) $B[k]$

for $i = 0$ to $n - 1$ **do**

for each entry e in sequence $B[i]$ **do**

 remove e from $B[i]$ and insert it at the end of S

Code Fragment 12.7: Bucket-sort.

It is easy to see that bucket-sort runs in $O(n + N)$ time and uses $O(n + N)$ space. Hence, bucket-sort is efficient when the range N of values for the keys is small compared to the sequence size n , say $N = O(n)$ or $N = O(n \log n)$. Still, its performance deteriorates as N grows compared to n .

An important property of the bucket-sort algorithm is that it works correctly even if there are many different elements with the same key. Indeed, we described it in a way that anticipates such occurrences.

Stable Sorting

When sorting key-value pairs, an important issue is how equal keys are handled. Let $S = ((k_0, v_0), \dots, (k_{n-1}, v_{n-1}))$ be a sequence of such entries. We say that a sorting algorithm is **stable** if, for any two entries (k_i, v_i) and (k_j, v_j) of S such that $k_i = k_j$ and (k_i, v_i) precedes (k_j, v_j) in S before sorting (that is, $i < j$), entry (k_i, v_i) also precedes entry (k_j, v_j) after sorting. Stability is important for a sorting algorithm because applications may want to preserve the initial order of elements with the same key.

Our informal description of bucket-sort in Code Fragment 12.7 guarantees stability as long as we ensure that all sequences act as queues, with elements processed and removed from the front of a sequence and inserted at the back. That is, when initially placing elements of S into buckets, we should process S from front to back, and add each element to the end of its bucket. Subsequently, when transferring elements from the buckets back to S , we should process each $B[i]$ from front to back, with those elements added to the end of S .

Radix-Sort

One of the reasons that stable sorting is so important is that it allows the bucket-sort approach to be applied to more general contexts than to sort integers. Suppose, for example, that we want to sort entries with keys that are pairs (k, l) , where k and l are integers in the range $[0, N - 1]$, for some integer $N \geq 2$. In a context such as this, it is common to define an order on these keys using the **lexicographic** (dictionary) convention, where $(k_1, l_1) < (k_2, l_2)$ if $k_1 < k_2$ or if $k_1 = k_2$ and $l_1 < l_2$ (see page 363). This is a pairwise version of the lexicographic comparison function, which can be applied to equal-length character strings, or to tuples of length d .

The **radix-sort** algorithm sorts a sequence S of entries with keys that are pairs, by applying a stable bucket-sort on the sequence twice; first using one component of the pair as the key when ordering and then using the second component. But which order is correct? Should we first sort on the k 's (the first component) and then on the l 's (the second component), or should it be the other way around?

To gain intuition before answering this question, we consider the following example.

Example 12.5: Consider the following sequence S (we show only the keys):

$$S = ((3, 3), (1, 5), (2, 5), (1, 2), (2, 3), (1, 7), (3, 2), (2, 2)).$$

If we sort S stably on the first component, then we get the sequence

$$S_1 = ((1, 5), (1, 2), (1, 7), (2, 5), (2, 3), (2, 2), (3, 3), (3, 2)).$$

If we then stably sort this sequence S_1 using the second component, we get the sequence

$$S_{1,2} = ((1, 2), (2, 2), (3, 2), (2, 3), (3, 3), (1, 5), (2, 5), (1, 7)),$$

which is unfortunately not a sorted sequence. On the other hand, if we first stably sort S using the second component, then we get the sequence

$$S_2 = ((1, 2), (3, 2), (2, 2), (3, 3), (2, 3), (1, 5), (2, 5), (1, 7)).$$

If we then stably sort sequence S_2 using the first component, we get the sequence

$$S_{2,1} = ((1, 2), (1, 5), (1, 7), (2, 2), (2, 3), (2, 5), (3, 2), (3, 3)),$$

which is indeed sequence S lexicographically ordered.

So, from this example, we are led to believe that we should first sort using the second component and then again using the first component. This intuition is exactly right. By first stably sorting by the second component and then again by the first component, we guarantee that if two entries are equal in the second sort (by the first component), then their relative order in the starting sequence (which is sorted by the second component) is preserved. Thus, the resulting sequence is guaranteed to be sorted lexicographically every time. We leave to a simple exercise (R-12.19) the determination of how this approach can be extended to triples and other d -tuples of numbers. We can summarize this section as follows:

Proposition 12.6: Let S be a sequence of n key-value pairs, each of which has a key $(k_1, k_2, \dots, k_d)$, where k_i is an integer in the range $[0, N - 1]$ for some integer $N \geq 2$. We can sort S lexicographically in time $O(d(n + N))$ using radix-sort.

Radix-sort can be applied to any key that can be viewed as a composite of smaller pieces that are to be sorted lexicographically. For example, we can apply it to sort character strings of moderate length, as each individual character can be represented as an integer value. (Some care is needed to properly handle strings with varying lengths.)